

Advanced Programming Exam

Introduction:

In the following I explain general concerns and implementation decisions task by task. When referring to tests that are part of (nested) testGroups, I denote them as following: $testGroup \rightarrow testSubgroup \rightarrow \dots \rightarrow testName$.

Task A:

I believe my solution for Task A to be fully functional. The behavior of each of the four introduced expressions is tested in a separate testGroup as part of the taskATests. For these tests I implemented a partial interpreter "testEval" (in Tests.hs) that is only able to handle the Pure and the ErrorOp case of the EvalM monad and returns an error in any other case. The other cases are not relevant for (the tests for) Task A. Each group includes at least one test where an error is expected. Mostly I only test whether an error is thrown, not which one, to be less dependent on specific error messages.

The task instruction remains relatively open regarding which properties a ValSet and its operations should fulfill. There were multiple ambiguities/cases not fully defined by the task:

Sets containing different types of Vals. It wasn't explicitly stated whether sets with mixed types of Vals should be allowed. E.g. a set containing a ValBool and a ValInt. In my implementation mixed type sets are explicitly allowed, as validated by the test $Task\ A \rightarrow Set \rightarrow Set\ with\ various\ types\ of\ Vals\ (including\ nested\ sets)$.

Duplicates. As sets usually imply to only include unique elements, my implementation of set construction and set union removes duplicate elements such that no ValSet contains duplicates at any point. I validated this behavior with multiple tests, including $Task\ A \rightarrow Set \rightarrow Set\ with\ duplicates$ and $Task\ A \rightarrow Union \rightarrow Union\ of\ overlapping\ sets$.

A downside of the duplicate removal is the overhead in terms of runtime. My current implementation of duplicate removal works by sorting the list and afterwards removing adjacent duplicates. Therefore the runtime lies in $O(n \log n)$ for a list of length n .

Set Equality. The Task does not specify whether/to which extend we need to implement the equality of sets. The duplicate-removal during set construction and union has the side effect of sorting the lists, thereby making the default derived list-equality accurate as a set-equality. However, this is a rather implicit property that I do not want to rely on. Future extensions, e.g. adding single elements to a set would need to be implemented while keeping in mind this very implicit assumption of keeping the list in order.

Therefore, I explicitly implemented the case for ValSet for the Eql expression. Here, I do not assume the lists to be sorted and determine the equality based on whether the two sets contain the same elements (order independent). This is validated in the test $Task\ A \rightarrow Set \rightarrow Set\ Equality\ is\ order-independent$. What I did not do is overwrite the derived Eq behavior of ValSets, which by default just compares the equality of the lists and is therefore order dependent. I considered changing this to be out of scope for this exam.

Removal of non-existing elements. The task does not explicitly specify what should happen when the second operand of Without contains elements that are not contained in the first operand.

I decided to not throw an error and just ignore these elements. I validated this behavior in the test *Task A $\rightarrow$ Without $\rightarrow$ Without where some elements are in second set but not in first set.*

Task B:

I believe my solution for Task B to be fully functional. I implemented multiple tests in testGroup *Task B*, including tests validating general behavior, left/right associativity and operator precedence.

The task description is mostly non-ambiguous, I just want to mention minor details:

Whitespaces in Set Constructor. It was not specified whether whitespaces are valid between the braces in an empty set construction. I decided to allow whitespaces, which I validated in the Test *TaskB $\rightarrow$ Set Construction $\rightarrow$ { }*.

Validation that Set operations are applied on sets. At the parsing stage, I do not validate whether Set operations are actually applied on sets. E.g. test *Task B $\rightarrow$ Without and Union Operator $\rightarrow 1 + 2 \text{ union } 2$* shows that the parser accepts this "invalid" union of a ValInt and a ValSet and returns an Exp that would return an error upon evaluation. This validation is in my opinion not part of the parsing stage.

Task C:

I believe my solution for Task C to be fully functional. The tests in taskCtests demonstrate that my concurrent implementation produces the same results as BlockAutomaton.Simulation given the smoothen rule-set and the default-galapagos rule-set.

Model. I implemented the following general model:

There is one supervisor server and one thread for each block which I call "block-worker". Each block-worker has 5 input channels: one for receiving messages from the supervisor server and one for receiving messages from each neighboring block. The following messages are supported:

Flow	Notes
External $\rightarrow$ Supervisor Server	MsgTriggerStep MsgObserve (ReplyChan [[obs]])
Supervisor Server $\rightarrow$ Block-Worker	MsgPerformStep Bool MsgObserveBlock (ReplyChan (CellStates obs))
Block-Worker $\rightarrow$ Block-Worker	MsgShiftBlock state state

Step. When a Step is triggered on the supervisor server, each block-worker is notified to perform a step. On the block-worker level, a step consists of three operations: First, the interactions between the cells within the block are performed. Then one horizontal and one vertical "shift" move the cells between the workers s.t. afterwards the block-workers each have the margolus-neighborhood for the next step. A shift means moving all cells in the grid in one direction. E.g. a right-shift means that the upper-left and lower-left cells get shifted to be the upper-right and lower-right cells respectively while the previous upper- and lower-right cells get send to the neighbor to the right where they replace the previous two left cells. Importantly each shift starts with the sending operation. Only after sending a block-worker reads from the respective input-channel.

Observe. The Observe operation is implemented very straightforward: The supervisor-server requestReplies the 4 observations on cells of every block and combines them to one large grid that is returned.

Channel-initialization. One part of my implementation that I want to highlight is the initialization of the communication channels. In the beginning, every block-worker needs to be assigned input and output channels to communicate with its neighbors. Each input-channel of a block needs to be assigned as the output-channel of a neighboring block. The simplest way to achieve this is to

create all input channels and then add output-channels by doing index-based accesses on the grid of input channels.

I wanted to avoid any index-based accesses on list as they are inefficient (and also non-idiomatic). Therefore I derived my grid of output-channels by rotating the individual directions of my grid of input channels. E.g. the "right-direction" input channels rotated once to the right result in the "left-direction" output-channels.

```
initializeChannels :: Int -> Int -> IO [[(NeighborChannels state obs, NeighborChannels state obs)]]
initializeChannels numBlockRows numBlockCols = do
  topInputs <- replicateM numBlockRows $ replicateM numBlockCols newChan
  bottomInputs <- replicateM numBlockRows $ replicateM numBlockCols newChan
  leftInputs <- replicateM numBlockRows $ replicateM numBlockCols newChan
  rightInputs <- replicateM numBlockRows $ replicateM numBlockCols newChan

  let topOutputs = rotateForward bottomInputs
      bottomOutputs = rotateBack topInputs
      leftOutputs = rotateRight rightInputs
      rightOutputs = rotateLeft leftInputs

      inputChans = zipChannels topInputs bottomInputs leftInputs rightInputs
      outputChans = zipChannels topOutputs bottomOutputs leftOutputs rightOutputs

  pure $ zipWith (zipWith (\i o -> (i, o))) inputChans outputChans
```

Monadic Server State. Another detail worth mentioning is the implementation of the internal state of the block-worker, for which I used a state-monad. The benefit over a non-monadic solution is rather small in this case. An example where the advantages of the monadic solution shows is the implementation of perform-step which can be written like this

```
MsgPerformStep isEven -> do
  performInteractions
  cellShift isEven Horizontal
  cellShift isEven Vertical
  bwMainMsgLoop chan
```

instead of

```
MsgPerformStep isEven -> do
  let ibws' = performInteractions ibws
  ibws'' <- cellShift ibws' isEven Horizontal
  ibws''' <- cellShift ibws'' isEven Vertical
  bwMainMsgLoop ibws''' chan
```

in a non-monadic solution where the state needs to be explicitly passed.

The state of the supervisor-server is maintained non-monadic because it doesn't get updated a lot after initialization. Especially, we do not have any cases of multiple incremental state updates for the supervisor-server.

Task D:

I believe my implementation of Task D to be fully functional. To validate this I added various tests covering the basic meeting scenarios and some edge-cases. The latter include *Task D* $\rightarrow$ *Simple successful meets* $\rightarrow$ *MaxHP can not be exceeded*, *Task D* $\rightarrow$ *Invalid meets* $\rightarrow$ *Valid MeetOp*, *invalid*

interaction and Task D $\rightarrow$ *Multiple meets (strategy updates)* $\rightarrow$ *meeting with invalid finch does not update strategy.*

There is one small ambiguity in the task description. It states that in a scenario where finchA behaves illegally, finchB should not be modified unless it is also invalid. The task does not specify how actively we need to check for the invalidity of finchB. Lets say finchA immediately fails because the strategy does not produce a valid MeetOp. Are we supposed to evaluate the strategy of finchB to a point where we check whether there is a valid MeetOp followed by a valid GroomOp/IgnoreOp? In my implementation I am actively checking whether both finches are behaving completely valid, even if one finch fails early.

Task E:

I believe my implementation of Task E to be fully functional, including support for arbitrary ordering of params. My tests implemented in *Task E* validate the behavior for various valid and invalid inputs, including tests for the order of parameters.

I resolved minor ambiguities of the task in the following way:

Last write wins. In case one string to be parsed includes multiple occurrences of the same param, the last occurrence is the one that gets saved.

+ not supported. While an integer was explicitly defined to support a "-" sign, the task does not say anything about a "+". I decided not to allow them as it is not that common to use explicit "+" signs for positive numbers. Therefore e.g. the input-string "height=+10 width=20" fails which I validated in a test.

Leading character of strategy name. Strategy names are generally allowed to contain letters, numbers and a few special characters. The task did not state whether the names are allowed to start with any of these characters as well. In my implementation, any of these characters are allowed as the leading character of a name, e.g. making "_samaritan-2_0.v1" a valid strategy name.

Task F:

I believe my solution of Task F to be fully functional.

For this task I did not encounter any ambiguities.

Testing the system holistically proved difficult, mainly due to the randomness during initialization and reproduction. In the end I did some manual testing by using the renderGrid-function from viewer.hs to visualize the grid over multiple steps and manually asserting that each transition to the next step is plausible.

Question Answering

Question 1: The final grammar (including "***") looks as follows:

```
Atom ::= var
| int
| bool
| "(" Exp ")"
| "meet"
| "ignore"
| "groom"
| "{" "}"
| "{" Exp Exps "}"
```

```
Exps ::= "," Exp Exps
```

```

|

FExp' ::= Atom FExp'
|

FExp ::= Atom FExp'

LExp ::= FExp
| "if" Exp "then" Exp "else" Exp
| "\"var ">" Exp
| "let" var "=" Exp "in" Exp
| "loop" var "=" Exp "for" var "<" Exp "do" Exp

Exp3' ::= "**" LExp Exp3'
|

Exp3 ::= LExp Exp3'

Exp2' ::= "*" Exp3 Exp2'
| "/" Exp3 Exp2'
|

Exp2 ::= Exp3 Exp2'

Exp1' ::= "+" Exp2 Exp1'
| "-" Exp2 Exp1'
|

Exp1 ::= Exp2 Exp1'

ExpSet1' ::= "union" Exp1 ExpSet1'
| "without" Exp1 ExpSet1'
|

ExpSet1 ::= Exp1 ExpSet1'

ExpSet0' ::= "elem" ExpSet1 ExpSet0'
|

ExpSet0 ::= ExpSet1 ExpSet0'

Exp0' ::= "==" Exp1 Exp0'
|

Exp0 ::= Exp1 Exp0'

Exp ::= Exp0

```

My parser implementation is mostly mirroring the grammar. Every nonterminal is a function (usually the name of the nonterminal with the prefix "p"). Any nonterminal NT' is implemented as a "chain" sub-function in the pNT function.

The only point where I derive from directly mirroring the grammar are the set-constructors. They can be implemented very closely to the grammar e.g. by adding

```
try $ lString "{" *> lString "}" *> pure (Set []),
try $ lString "{" *> (Set <$> ((:) <$> pExp <*> pExps)) <*> lString "}",
```

to the choices of pAtom and adding the function

```
pExps :: Parser [Exp]
pExps = choice [
  do
    y <- lString "," *> pExp
    ys <- pExps
    pure $ y : ys,
  pure []
]
```

. However, all of this can be replaced just by adding

```
lString "{" *> (Set <$> sepBy pExp (lString ",") <*> lString "}"),
```

to the choices of pAtom. Both implementations are equivalent in their behavior. My final handin includes the latter implementation, as it is much more concise.

Question 2: The memory consumption of a set that is repeatedly unioned with itself does not increase. I implemented the union operation such that it removes duplicate elements while unioning, thereby not increasing the size of the set with each iteration.

```
eval env (Union e1 e2) = do
  v1 <- eval env e1
  v2 <- eval env e2
  case (v1, v2) of
    (ValSet s1, ValSet s2) ->
      pure $ ValSet $ removeDuplicatesFast (s1 ++ s2)
    (_, _) ->
      failure "Unexpected Val-type. Union only works on ValSets."
```

Depending on the loop implementation, running large loops can generally lead to increased memory consumption if the env gets incrementally extended with the running variables i and p . However, this is not the case for the loop implementation given in the handout.

Question 3: I am not sure whether "a finch strategy going into an infinite loop" refers to

- the eval of the finch-strategy expression being stuck in an infinite loop or
- the continuation of a MeetOp/GroomOp/IgnoreOp being stuck in an infinite loop.

In either case, the current behavior of my implementation and the solution are similar.

Current behavior. I do not handle the case of possible infinite loops. Therefore this loop would infinitely block the worker trying to initialize the finch/perform the meeting-interaction.

Solution. A better way to handle this would be to externalize the respective function that is possibly running into an infinite loop (in the following referred to as function F) to a new thread while adding a timeout.

More specifically, we would need to create a channel C that allows two types of messages: a successful result of F or a timeout signal. Then we would need to spawn two new threads, passing to each this channel C . One of the threads tries to execute F and sends the result to the channel C , the other

one sleeps for a predetermined timeout-period and then sends a timeout signal to channel C. The worker that spawns the two threads reads from channel C. In case the timeout signal arrives before the thread executing F finishes, the cell gets initialized as empty/the behavior of the finch is declared as invalid and the cell gets cleared. Afterwards, both external threads get killed.

Question 4: (I explain my general model in the introduction in part "Task C")

I am fairly confident that my implementation of Task C does not have any race conditions. One property prohibiting race conditions, is that each channel is read by only one thread and written to by only one thread. This already eliminates a lot of potential for race conditions. Another important point is that the communication between neighboring block-workers is not done through multiple iterations of the main message loop but is instead a process that blocks everything else until it is finished:

- When a step is executed on a block-worker, this is blocking until the step-execution is finished. Any further signals from the supervisor-server e.g. triggering further steps do not get read until the step is completely finished.
- Similar properties hold within the step:
 - First interactions within the block are performed.
 - Only when that is finished, the horizontal shift is triggered.
 - Only when that is finished, the vertical shift gets executed.
 - Note that each shift is implemented s.t. each worker first sends and only after that receives from the neighboring block.

Question 5: (I explain my general model in the introduction in part "Task C")

Each step consists of the following messages:

- One message to the supervisor-server.
- One message from the supervisor-server to each block-worker triggering the step.
- Two shifts, where each shift consists of one message per block-worker.

In total this makes $3 \cdot \#numberBlocks + 1$ messages per step.

In my model there are no messages required to perform the interaction between cells, as each neighborhood is handled by one block-worker. The four interactions between cells within a block get executed sequentially, but all blocks can get handled concurrently.

All messages within a shift are sent and received concurrently. The two shifts get executed sequentially. This does not mean that a global synchronization across all blocks is necessary, but a block-worker needs to finish the horizontal shift before being able to start with the vertical shift, while being dependent only a single immediate neighbor per shift.

Question 6: The simplest way to implement this would be to pass the strategy-object of the parent to the child.

The free monads that are used to store the strategy contain a finite amount of MeetOps. In the given examples, e.g. in Tests.hs or in example.rules they are usually initialized with a for-loop with 100 iterations. Usually this limitation is not an issue, as the lifespan of a finch runs out before an finch can have this many meets. However, when just copying when reproducing, a child finch (that gets initialized with the full lifespan) continues the stack of MeetOps that already progressed quite a few iterations beforehand. Even when initializing the strategy with a large number of MeetOps, at some point after multiple "generations" cases can occur where the strategy runs out of MeetOps and the free monad returns a Pure value. This would lead to invalid behavior and therefore removal of the finch.

Question 7: QuickCheck can be utilized to test arbitrary parameter ordering. In my implementation, I generate multiple strings with arbitrary parameter order based on the same individual parameter definitions. The parsing of each of these randomly ordered strings needs to return the same config.

```
testCase "Arbitrary Order Parsing" $ do
  samples <- replicateM 10 (generate genArbitraryOrderInputString)
  forM_ samples $ \s ->
    case parseGalapagos "input" s of
      Left err -> assertFailure err
      Right e' -> e' @?= parsedCfg
```

The way I implemented the generation of the input strings, is by having a list of Strings where each string defines one param. For each run, I shuffle the list and concatenate the strings:

```
genArbitraryOrderInputString :: Gen String
genArbitraryOrderInputString = do
  wh <- concat <$> shuffle whStrings
  params <- concat <$> shuffle paramStrings
  pure (wh ++ params)
```